Praktikum 4

NAMA	: Ihsanta Zaki Sanjaya
KELAS	: 2 D4 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Prof. Ir. Dadet Pramadihanto, M.Eng, Ph.D
MATA KULIAH	: Praktikum Pengolahan Sinyal dan Citra
TANGGAL	: Kamis, 13 Maret 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 4
Discrete Fourier Transform

1. DASAR TEORI

DFT adalah teknik fundamental dalam pengolahan sinyal digital yang memungkinkan analisis karakteristik frekuensi dari urutan data terbatas. Berbeda dengan transformasi Fourier kontinu, DFT melakukan dekonstruksi sinyal menjadi komponen frekuensi diskret, membagi spektrum menjadi sejumlah terbatas titik sampling. Proses ini mengkonversi informasi sinyal dari domain temporal menjadi representasi spektral yang memungkinkan identifikasi dan analisis komponen frekuensi dasar dalam sinyal digital.

DTF dari suatu sinyal diskrit x[n] dinyatakan dalam bentuk persamaan sebagai berikut:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \ldots, N-1$$

DFT memiliki hubungan erat dengan DTFT, di mana DFT dapat dianggap sebagai versi diskrit dari DTFT yang diambil sampelnya pada N titik frekuensi yang merata antara 0 dan 2π. Karena itu, DFT sering digunakan dalam aplikasi dunia nyata, terutama karena kebanyakan sinyal digital memiliki panjang terbatas. Beberapa properti penting dari DFT di antaranya adalah periodisitas, simetri kompleks, dan linearitas. Periodisitas berarti hasil DFT memiliki pola berulang dengan periode N, karena DFT mengasumsikan bahwa sinyal input adalah periodik dengan periode N. Sementara itu, simetri kompleks menyatakan bahwa jika sinyal input adalah real, maka separuh nilai DFT dapat ditentukan dari separuh lainnya, yang dapat menghemat perhitungan.

Dalam implementasinya, DFT dapat dihitung menggunakan Python dengan numpy, contohnya dengan fungsi np.fft.fft(). Namun, perhitungan langsung DFT memiliki kompleksitas $O(N^2)$, yang membuatnya lambat untuk sinyal panjang. Untuk mengatasinya, digunakan Fast Fourier Transform (FFT), yaitu algoritma yang mengoptimalkan komputasi DFT dan mengurangi kompleksitas menjadi O(N log N). Salah satu metode FFT yang paling umum adalah algoritma Cooley-Tukey, yang menggunakan pendekatan *divide and conquer* dengan membagi DFT menjadi sub-DFT lebih kecil hingga tersisa operasi sederhana. Jika panjang sinyal N merupakan kelipatan 2, algoritma Radix-2 FFT dapat digunakan untuk mempercepat perhitungan lebih lanjut. Terdapat banyak pengaplikasiannya, terutama dalam analisis spektrum, pemrosesan sinyal digital, dan kompresi data. Dalam analisis spektrum, FFT memungkinkan kita mengidentifikasi frekuensi dominan dalam sinyal, misalnya dalam analisis suara atau radar. Dalam filtering digital, FFT mempercepat perhitungan konvolusi, sehingga lebih efisien dibandingkan melakukannya langsung di domain waktu. Dalam kompresi data, FFT juga membantu menghilangkan komponen frekuensi yang tidak signifikan untuk mengurangi ukuran file, seperti yang digunakan dalam format JPEG untuk gambar atau MP3 untuk audio.

.

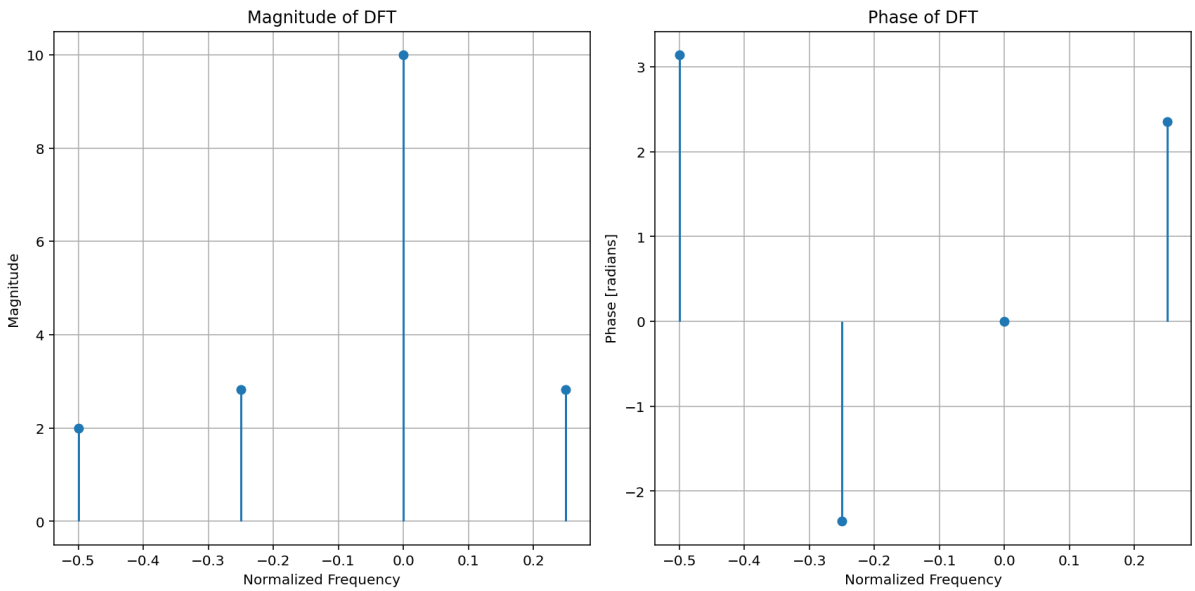
2. PROSEDUR

3.4.1 Computing the DFT

Program:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4])
X = np.fft.fft(x)
N = len(x)
freqs = np.fft.fftfreq(N)

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.stem(freqs, np.abs(X), basefmt=" ")
plt.title("Magnitude of DFT")
plt.xlabel("Normalized Frequency")
plt.ylabel("Magnitude")
plt.grid(True)

plt.subplot(1, 2, 2)
plt.stem(freqs, np.angle(X), basefmt=" ")
plt.title("Phase of DFT")
plt.xlabel("Normalized Frequency")
plt.ylabel("Phase [radians]")
plt.grid(True)

plt.tight_layout()
plt.show()
```

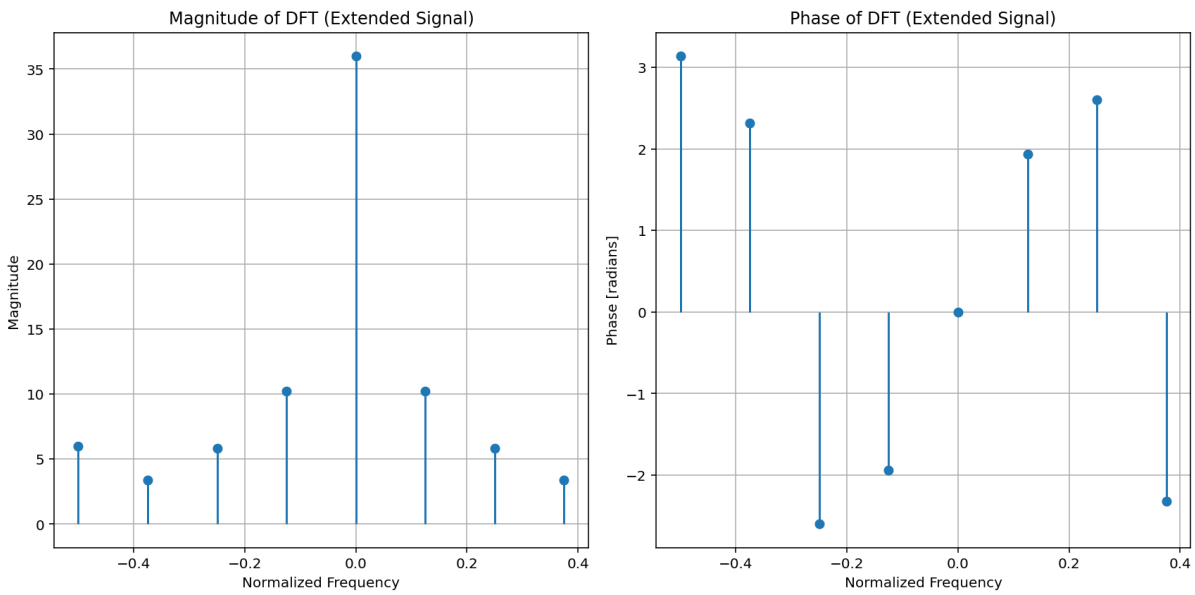
Hasil:



Variasi 1, mengubah panjang x[n] = {1,3,2,5,4,6,8,7}:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 3, 2, 5, 4, 6, 8, 7])
X = np.fft.fft(x)
N = len(x)
freqs = np.fft.fftfreq(N)

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.stem(freqs, np.abs(X), basefmt=" ")
plt.title("Magnitude of DFT (Extended Signal)")
plt.xlabel("Normalized Frequency")
plt.ylabel("Magnitude")
plt.grid(True)

plt.subplot(1, 2, 2)
plt.stem(freqs, np.angle(X), basefmt=" ")
plt.title("Phase of DFT (Extended Signal)")
plt.xlabel("Normalized Frequency")
plt.ylabel("Phase [radians]")
plt.grid(True)

plt.tight_layout()
plt.show()
```

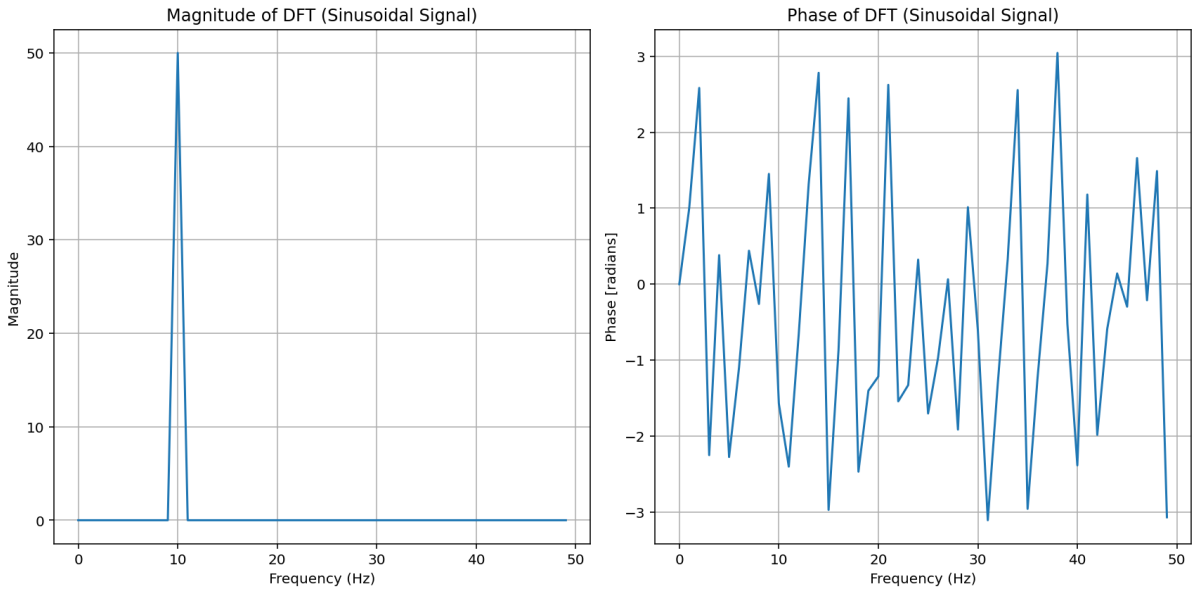
Hasil:



Variasi 2, menggunakan sinyal sinusoidal:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 100
t = np.arange(0, 1, 1/fs)
x = np.sin(2 * np.pi * 10 * t)

X = np.fft.fft(x)
N = len(x)
freqs = np.fft.fftfreq(N, 1/fs)

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.plot(freqs[:N//2], np.abs(X[:N//2]))
plt.title("Magnitude of DFT (Sinusoidal Signal)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(freqs[:N//2], np.angle(X[:N//2]))
plt.title("Phase of DFT (Sinusoidal Signal)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Phase [radians]")
plt.grid(True)

plt.tight_layout()
plt.show()
```

Hasil:



3.6.1 FFT

Program:

```
import numpy as np
import time

N = 1024
signal = np.random.randn(N)
start_time = time.time()
dft = np.fft.fft(signal)
fft_time = time.time() - start_time

print(f"Time taken by FFT: {fft_time:.6f} seconds")
```

Hasil:

```
dan Citra/3.6.1.py' --wdir
Time taken by FFT: 0.000000 seconds
```

Variasi 1, menggunakan lebih banyak sample dan penampilan waktu dalam 9 desimal:

```
import numpy as np
import time

N = 8192
signal = np.random.randn(N)

start_time = time.perf_counter()
dft = np.fft.fft(signal)
fft_time = time.perf_counter() - start_time

print(f"Time taken by FFT: {fft_time:.9f} seconds")
```

Hasil:

```
dan Citra/3.6.1v1.py' --wdir
Time taken by FFT: 0.000368000 seconds
```

3.6.3 Radix-2 FFT Algorithm

Program:

```
import numpy as np

def radix2_fft(signal):
    N = len(signal)
    if N <= 1:
        return signal

    even_part = radix2_fft(signal[0::2])
    odd_part = radix2_fft(signal[1::2])

    combined = [0] * N
    for k in range(N // 2):
        t = np.exp(-2j * np.pi * k / N) * odd_part[k]
        combined[k] = even_part[k] + t
        combined[k + N // 2] = even_part[k] - t

    return combined

signal = np.random.randn(8)
fft_result = radix2_fft(signal)
print("FFT Result:", fft_result)
```

Hasil:

```
Citra/3.6.3.py' --wdir
FFT Result: [np.complex128(-4.4564672420888884+0j),
np.complex128(-0.9095520502162769-4.187735246996334j),
np.complex128(0.09399431935460831+2.185558858489643j),
np.complex128(1.6397217769605175+3.5445876885411094j),
np.complex128(-1.7996726489174506+0j),
np.complex128(1.639721776960518-3.5445876885411094j),
np.complex128(0.09399431935460859-2.185558858489643j),
np.complex128(-0.9095520502162774+4.187735246996334j)]
```

Variasi 1, mengubah panjang sinyal 16:

```
import numpy as np

def radix2_fft(signal):
    N = len(signal)
    if N <= 1:
        return signal

    even_part = radix2_fft(signal[0::2])
    odd_part = radix2_fft(signal[1::2])

    combined = [0] * N
    for k in range(N // 2):
        t = np.exp(-2j * np.pi * k / N) * odd_part[k]
        combined[k] = even_part[k] + t
        combined[k + N // 2] = even_part[k] - t

    return combined

signal = np.random.randn(16)
fft_result = radix2_fft(signal)
print("FFT Result:", fft_result)
```

Hasil:

```
Citra/3.6.4av1.py' --wdir
FFT Result: [np.complex128(-1.3188435388109854+0j),
np.complex128(3.6338177887572014+4.9539737697973685j),
np.complex128(-3.2722636722973526+6.472724426502009j),
np.complex128(-0.9564507211573046+1.9151597472788866j),
np.complex128(-0.4393446906674238+2.735791096240754j),
np.complex128(3.505652804535803-3.844849785915023j),
np.complex128(-1.2831980830463832-0.2955414949281261j),
np.complex128(0.3134850834708127-0.6341590280120091j),
np.complex128(-5.850046082132244+0j),
np.complex128(0.31348508347081294+0.6341590280120091j),
np.complex128(-1.2831980830463827+0.2955414949281261j),
np.complex128(3.505652804535802+3.844849785915023j),
np.complex128(-0.4393446906674235-2.735791096240754j),
np.complex128(-0.9564507211573041-1.9151597472788875j),
np.complex128(-3.272263672297352-6.472724426502009j),
np.complex128(3.6338177887572023-4.953973769797368j)]
```

Variasi 2, menggunakan input sinyal sinousidal:

```python
import numpy as np

def radix2_fft(signal):
    N = len(signal)
    if N <= 1:
        return signal

    even_part = radix2_fft(signal[0::2])
    odd_part = radix2_fft(signal[1::2])

    combined = [0] * N
    for k in range(N // 2):
        t = np.exp(-2j * np.pi * k / N) * odd_part[k]
        combined[k] = even_part[k] + t
        combined[k + N // 2] = even_part[k] - t

    return combined

N = 8
t = np.arange(N)
signal = np.sin(2 * np.pi * t / N)
fft_result = radix2_fft(signal)
print("FFT Result for Sinusoidal Signal:", fft_result)
```

Hasil:

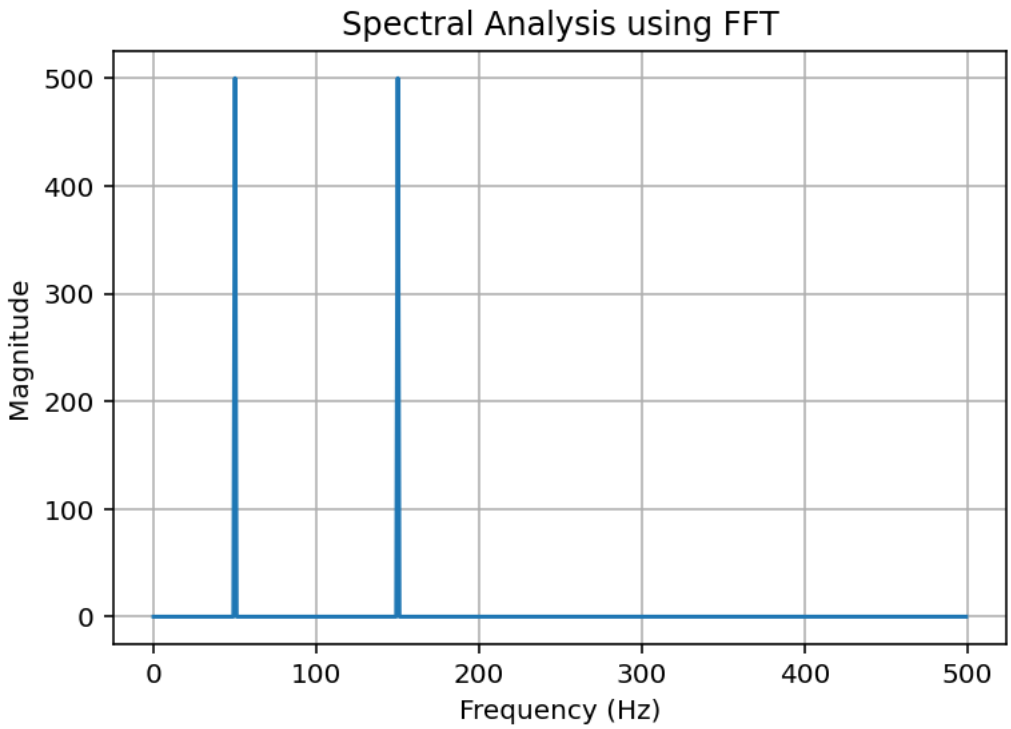
```
Citra/3.6.4av2.py' --wdir
FFT Result for Sinusoidal Signal:
[np.complex128(1.2246467991473532e-16+0j),
np.complex128(-2.220446049250313e-16-4j),
np.complex128(1.2246467991473535e-16-2.220446049250313e-16
j), np.complex128(1.9915985002059197e-16+0j),
np.complex128(1.2246467991473532e-16+0j),
np.complex128(2.220446049250313e-16+0j),
np.complex128(1.224646799147353e-16+2.220446049250313e-16j
), np.complex128(-6.890185696795333e-16+4j)]
```

3.6.4 Spectral Analysis

Program:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.arange(0, 1.0, 1.0 / fs)
freq1 = 50
freq2 = 150

signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t)
fft_result = np.fft.fft(signal)
freqs = np.fft.fftfreq(len(signal), 1 / fs)

plt.plot(freqs[:len(freqs) // 2], np.abs(fft_result)[:len(freqs) // 2])
plt.title("Spectral Analysis using FFT")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```

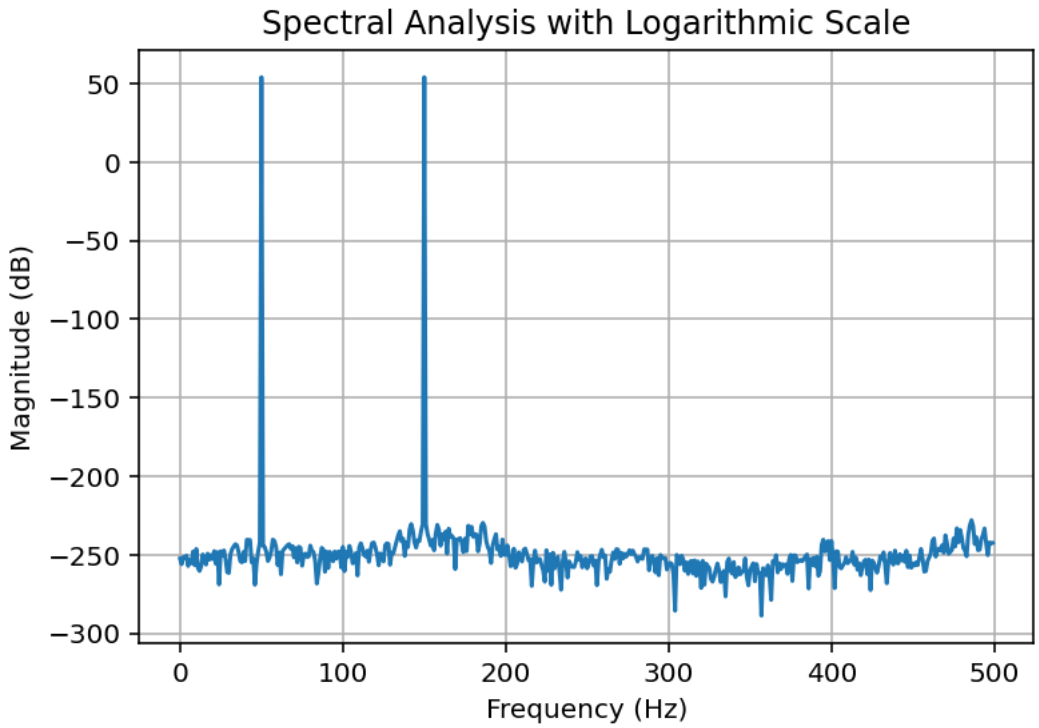
Hasil:



Variasi 1, menggunakan skala logaritmik:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.arange(0, 1.0, 1.0 / fs)
freq1 = 50
freq2 = 150

signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t)
fft_result = np.fft.fft(signal)
freqs = np.fft.fftfreq(len(signal), 1 / fs)
magnitude_db = 20 * np.log10(np.abs(fft_result)[:len(freqs) // 2])

plt.plot(freqs[:len(freqs) // 2], magnitude_db)
plt.title("Spectral Analysis with Logarithmic Scale")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (dB)")
plt.grid(True)
plt.show()
```

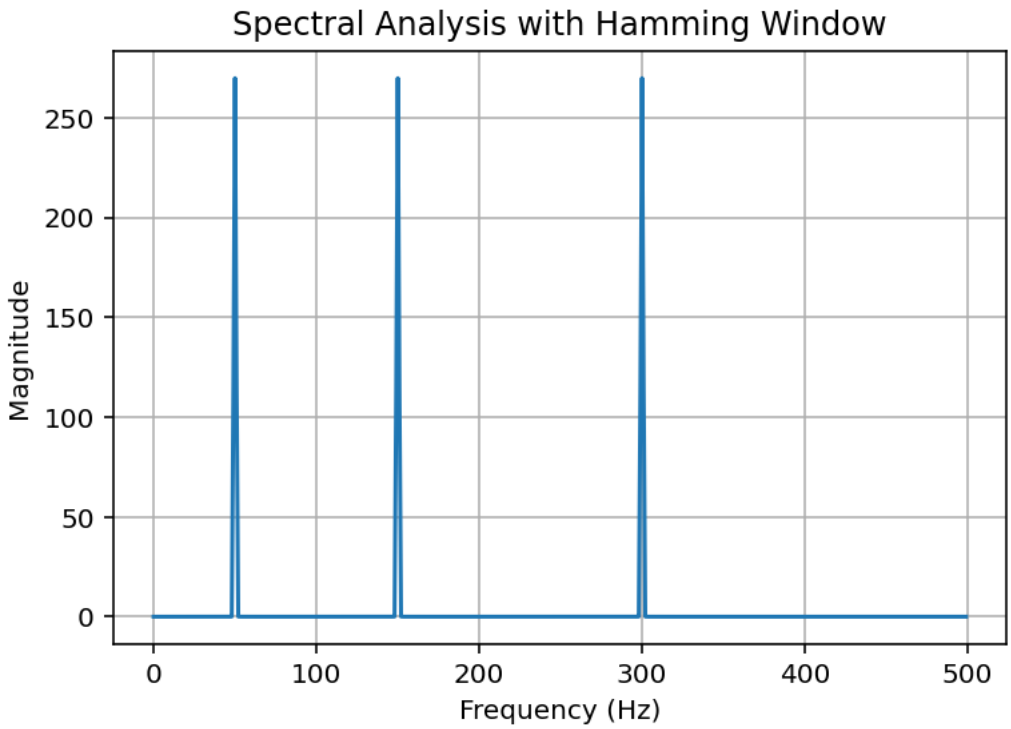
Hasil:



Variasi 2, menggunakan 3 frekuensi:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.arange(0, 1.0, 1.0 / fs)
freq1 = 50
freq2 = 150
freq3 = 300

signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t) + np.sin(2 * np.pi * freq3 * t)
window = np.hamming(len(signal))
windowed_signal = signal * window
fft_result = np.fft.fft(windowed_signal)
freqs = np.fft.fftfreq(len(signal), 1 / fs)

plt.plot(freqs[:len(freqs) // 2], np.abs(fft_result)[:len(freqs) // 2])
plt.title("Spectral Analysis with Hamming Window")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```

Hasil:

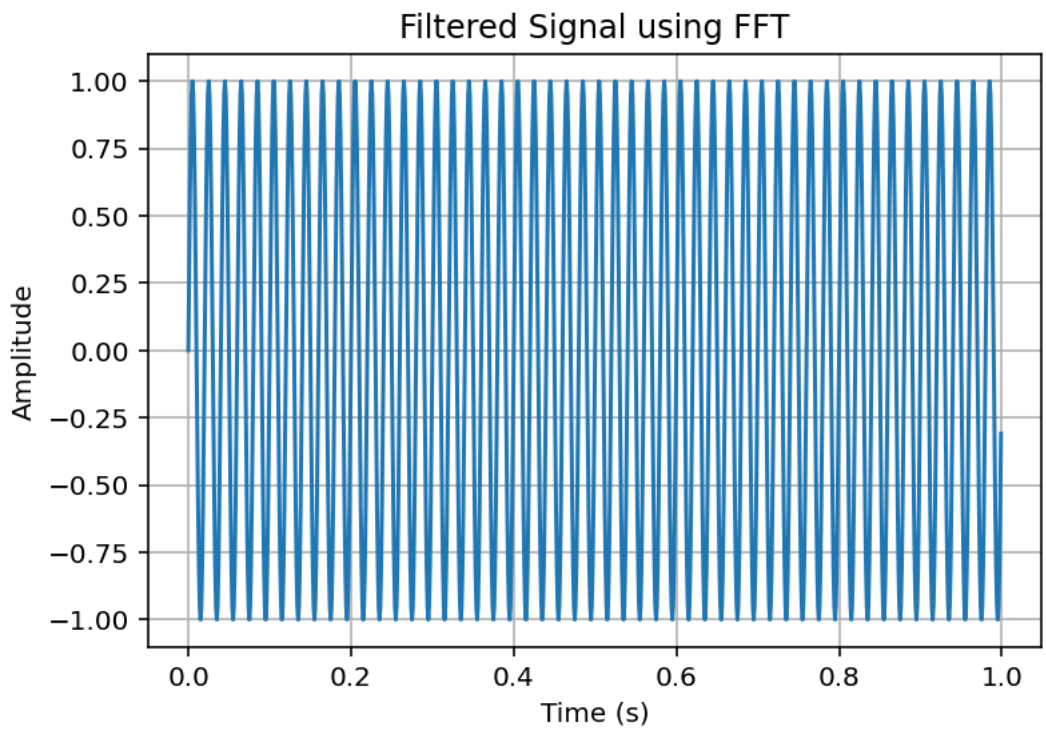


3.6.4 Filtering

Program:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.arange(0, 1.0, 1.0 / fs)
freq1 = 50
freq2 = 150

signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t)

cutoff_freq = 100
fft_signal = np.fft.fft(signal)
freqs = np.fft.fftfreq(len(signal), 1 / fs)
fft_signal[np.abs(freqs) > cutoff_freq] = 0
filtered_signal = np.fft.ifft(fft_signal)

plt.plot(t, np.real(filtered_signal))
plt.title("Filtered Signal using FFT")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

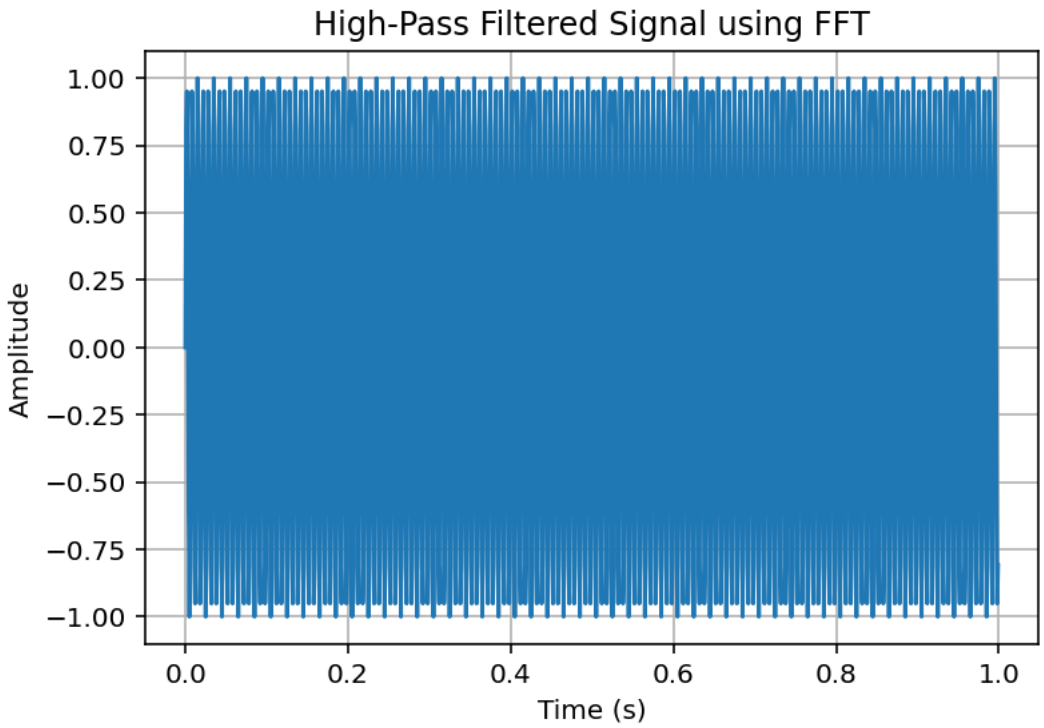
Hasil:



Variasi 1, menggunakan high pass filter:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.arange(0, 1.0, 1.0 / fs)
freq1 = 50
freq2 = 150

signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t)

cutoff_freq = 100
fft_signal = np.fft.fft(signal)
freqs = np.fft.fftfreq(len(signal), 1 / fs)
fft_signal[np.abs(freqs) < cutoff_freq] = 0
filtered_signal = np.fft.ifft(fft_signal)

plt.plot(t, np.real(filtered_signal))
plt.title("High-Pass Filtered Signal using FFT")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

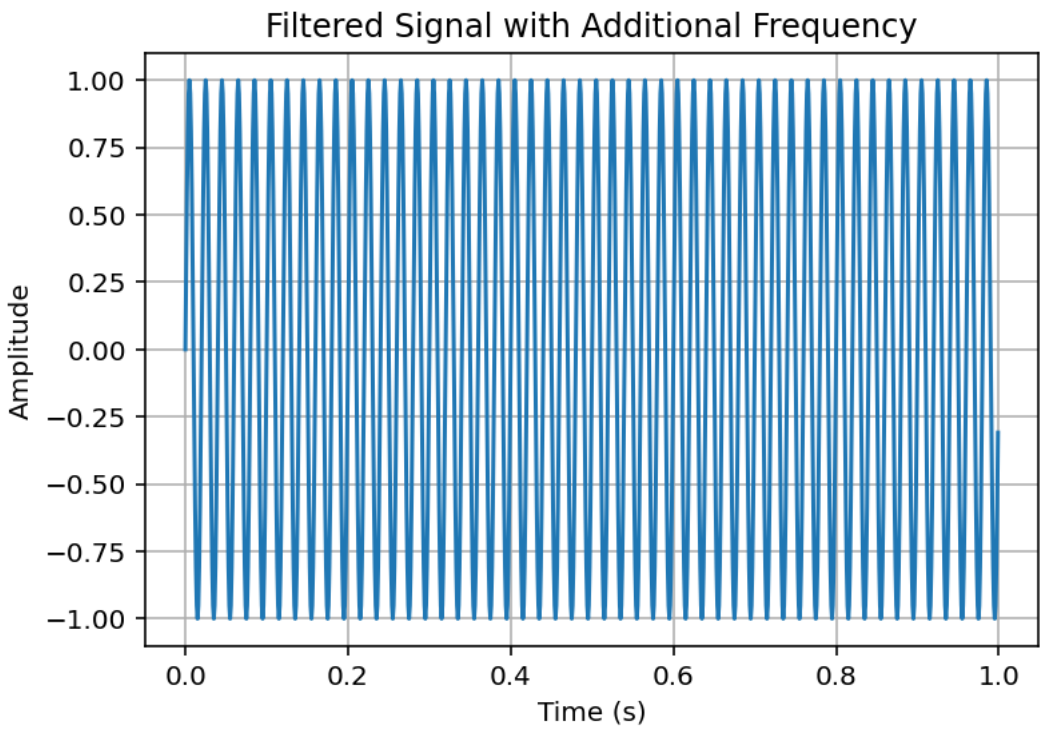
Hasil:



Variasi 2, menggunakan 3 frekuensi:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.arange(0, 1.0, 1.0 / fs)
freq1 = 50
freq2 = 150
freq3 = 250

signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t) + np.sin(2 * np.pi * freq3 * t)

cutoff_freq = 120
fft_signal = np.fft.fft(signal)
freqs = np.fft.fftfreq(len(signal), 1 / fs)
fft_signal[np.abs(freqs) > cutoff_freq] = 0
filtered_signal = np.fft.ifft(fft_signal)

plt.plot(t, np.real(filtered_signal))
plt.title("Filtered Signal with Additional Frequency")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

Hasil:

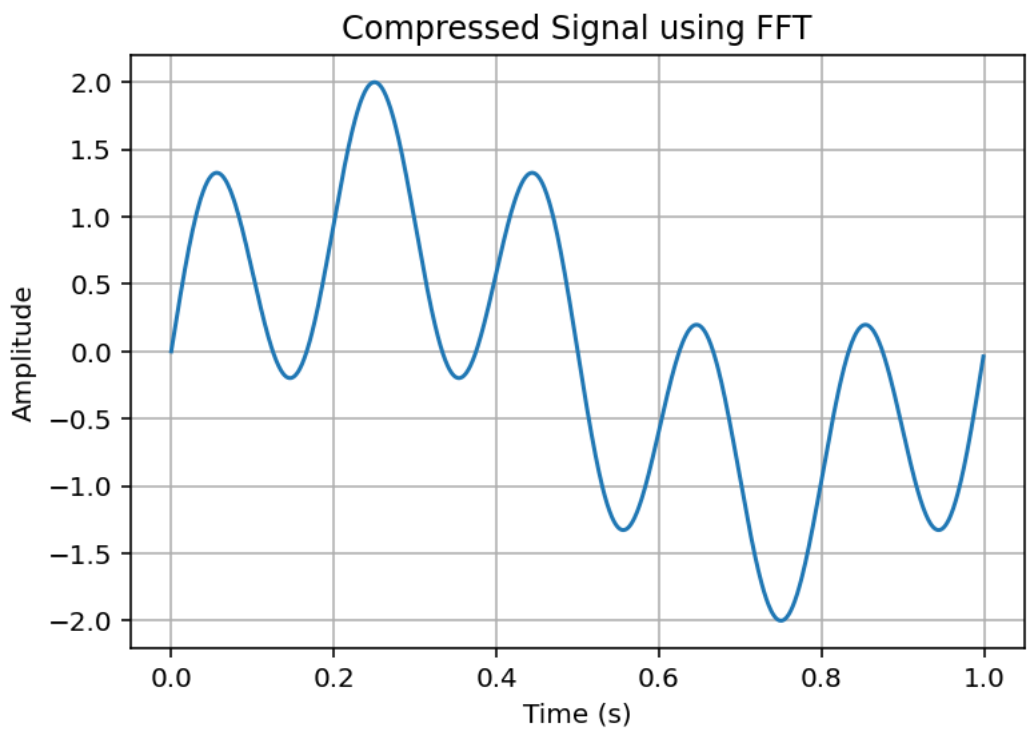


3.6.4 Signal Compression

Program:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.arange(0, 1.0, 1.0 / fs)
freq1 = 1
freq2 = 5

signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t)

fft_signal = np.fft.fft(signal)

threshold = 0.1 * np.max(np.abs(fft_signal))
fft_signal[np.abs(fft_signal) < threshold] = 0

compressed_signal = np.fft.ifft(fft_signal)

plt.plot(t, np.real(compressed_signal))
plt.title("Compressed Signal using FFT")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

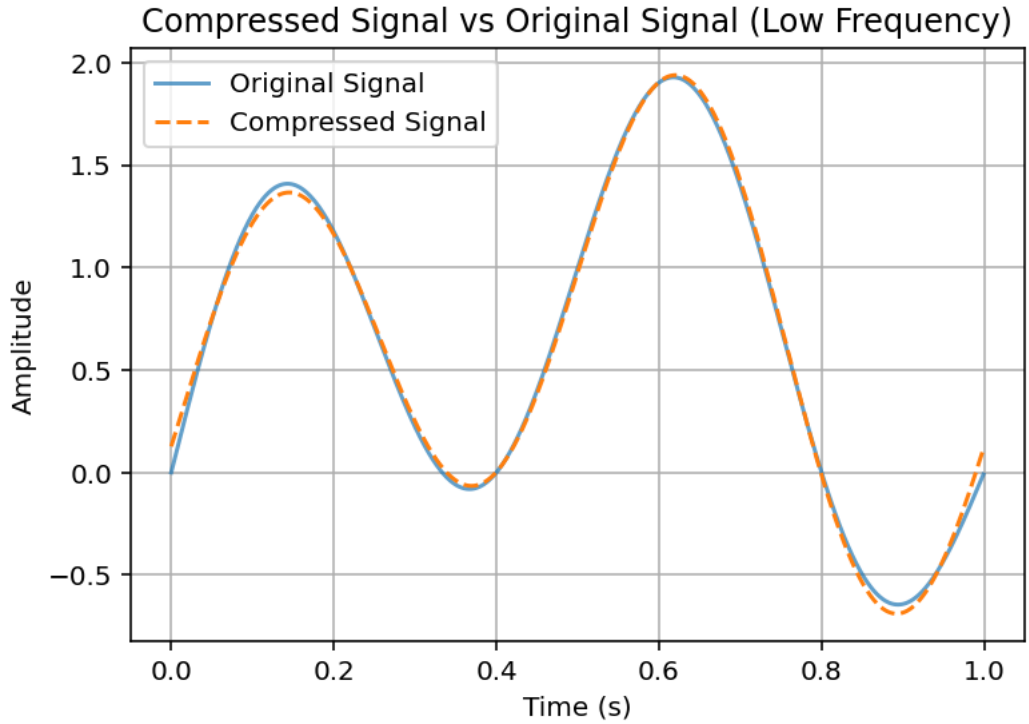
Hasil:



Variasi 1, menggunakan frekuensi yang berbeda:

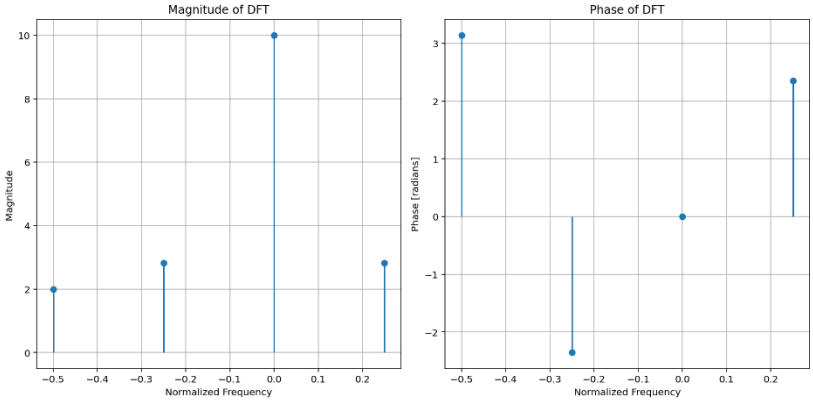
```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.arange(0, 1.0, 1.0 / fs)
freq1 = 0.5
freq2 = 2

signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t)

fft_signal = np.fft.fft(signal)

threshold = 0.1 * np.max(np.abs(fft_signal))
fft_signal[np.abs(fft_signal) < threshold] = 0

compressed_signal = np.fft.ifft(fft_signal)

plt.plot(t, signal, label="Original Signal", alpha=0.7)
plt.plot(t, np.real(compressed_signal), label="Compressed Signal", linestyle="dashed")
plt.title("Compressed Signal vs Original Signal (Low Frequency)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()
```

Hasil:

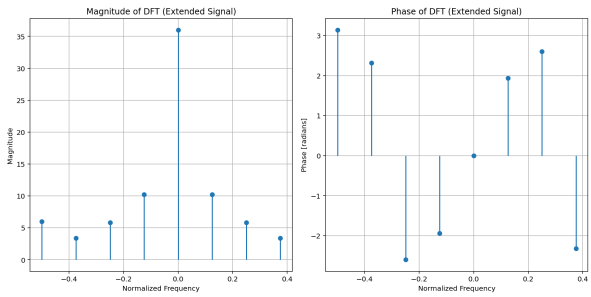


3. ANALISA

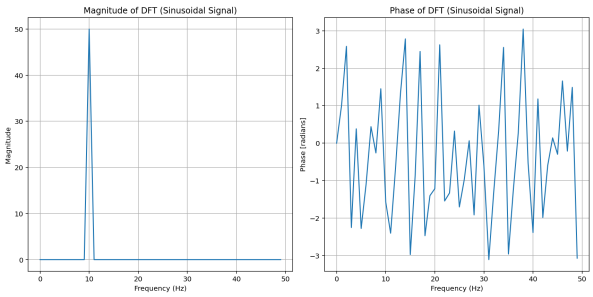
Pada praktikum ini, penerapan DFT dilakukan untuk menganalisis transformasi sinyal dari domain waktu ke domain frekuensi menggunakan Python. Program pertama menggunakan array sederhana [1, 2, 3, 4] untuk menghitung DFT. Dengan menggunakan fungsi np.fft.fft(), program menghasilkan representasi frekuensi dari sinyal masukan, menampilkan magnitude dan fase dari setiap komponen frekuensi melalui visualisasi stem plot.



Pada variasi pertama, panjang sinyal diperluas menjadi [1, 3, 2, 5, 4, 6, 8, 7], yang mengubah kompleksitas representasi frekuensi. Perubahan panjang sinyal menghasilkan pola magnitude dan fase yang berbeda, menunjukkan bagaimana struktur dan komposisi sinyal mempengaruhi karakteristik spektralnya. Stem plot memperlihatkan variasi amplitudo dan sudut fase pada setiap frekuensi diskret. Kemudian pada variasi kedua, menggunakan sinyal sinusoidal yang membawa kompleksitas analisis yang lebih tinggi. Program membangkitkan sinyal sinus dengan frekuensi 10 Hz, kemudian mentransformasikan ke domain frekuensi menggunakan DFT.



Var. 1



Var. 2

Percobaan selanjutnya penerapan FFT, percobaan ini mengeksplorasi efisiensi komputasional transformasi Fourier. Program awal menggunakan sinyal acak sepanjang 1024 sampel, mengukur waktu eksekusi transformasi menggunakan np.fft.fft(). Hasil menunjukkan kecepatan komputasi yang sangat tinggi, dengan waktu eksekusi yang mendekati nol detik.

```
dan Citra/3.6.1.py' --wdir
Time taken by FFT: 0.000000 seconds
```

Pada variasi pertama, ukuran sampel ditingkatkan menjadi 8192, dengan pengukuran waktu menggunakan time.perf_counter() untuk presisi lebih tinggi. Hasilnya menunjukkan peningkatan waktu komputasi yang masih sangat minimal, yaitu 0.000368 detik, menggarisbawahi efisiensi algoritma FFT.

```
dan Citra/3.6.1v1.py' --wdir
Time taken by FFT: 0.000368000 seconds
```

Percobaan selanjutnya mendemonstrasikan implementasi manual algoritma Radix-2 FFT, yang membagi sinyal menjadi komponen genap dan ganjil secara rekursif. Program awal menggunakan sinyal acak sepanjang 8 sampel, menerapkan transformasi Fourier dengan metode pembagian dan rekombinasi.

```
Citra/3.6.3.py' --wdir
FFT Result: [np.complex128(-4.4564672420888884+0j),
np.complex128(-0.9095520502162769-4.187735246996334j),
np.complex128(0.09399431935460831+2.185558858489643j),
np.complex128(1.6397217769605175+3.5445876885411094j),
np.complex128(-1.7996726489174506+0j),
np.complex128(1.639721776960518-3.5445876885411094j),
np.complex128(0.09399431935460859-2.185558858489643j),
np.complex128(-0.9095520502162774+4.187735246996334j)]
```

Pada variasi pertama, panjang sinyal ditingkatkan menjadi 16 sampel, menunjukkan skalabilitas algoritma. Variasi kedua menggunakan sinyal sinusoidal, yang menghasilkan hasil dengan nilai-nilai sangat kecil mendekati nol, mencerminkan sifat orthogonal dari transformasi Fourier.

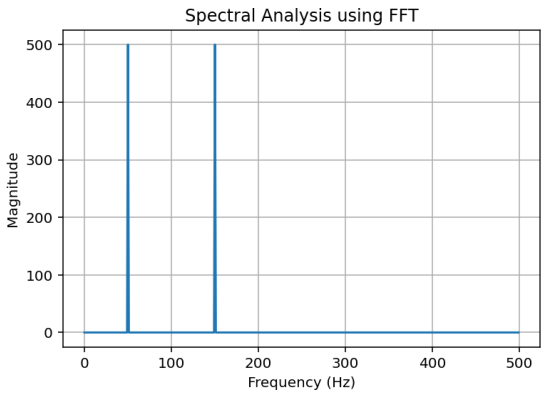
```
Citra/3.6.4av1.py' --wdir
FFT Result: [np.complex128(-1.3188435388109854+0j),
np.complex128(3.6338177887572014+4.9539737697973685j),
np.complex128(-3.2722636722973526+6.472724426502009j),
np.complex128(-0.9564507211573046+1.9151597472788866j),
np.complex128(-0.4393446906674238+2.735791096240754j),
np.complex128(3.505652804535803-3.844849785915023j),
np.complex128(-1.2831980830463832-0.2955414949281261j),
np.complex128(0.3134850834708127-0.6341590280120091j),
np.complex128(-5.850046082132244+0j),
np.complex128(0.3134850834708129+0.6341590280120091j),
np.complex128(-1.2831980830463827+0.2955414949281261j),
np.complex128(3.505652804535802+3.844849785915023j),
np.complex128(-0.4393446906674235-2.735791096240754j),
np.complex128(-0.9564507211573041-1.9151597472788875j),
np.complex128(-3.272263672297352-6.472724426502009j),
np.complex128(3.6338177887572023-4.953973769797368j)]
```

Var. 1

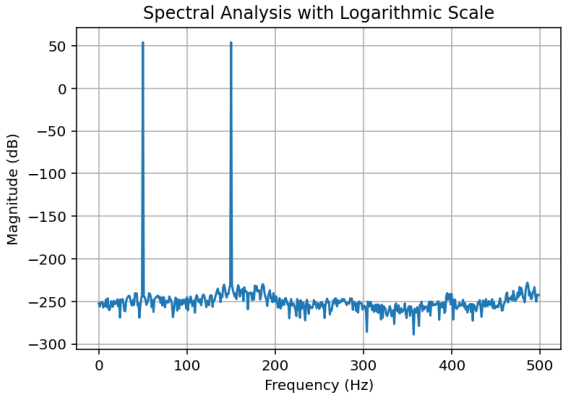
```
Citra/3.6.4av2.py' --wdir
FFT Result for Sinusoidal Signal:
[np.complex128(1.2246467991473532e-16+0j),
np.complex128(-2.220446049250313e-16-4j),
np.complex128(1.2246467991473535e-16-2.220446049250313e-16
j), np.complex128(1.9915985002059197e-16+0j),
np.complex128(1.2246467991473532e-16+0j),
np.complex128(2.220446049250313e-16+0j),
np.complex128(1.224646799147353e-16+2.220446049250313e-16j
), np.complex128(-6.890185696795333e-16+4j)]
```

Var. 2

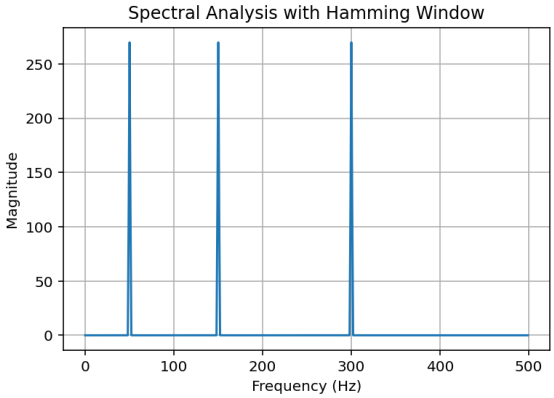
Percobaan selanjutnya menganalisis spektral, percobaan dilakukan dengan mensintesis sinyal multi-frekuensi pada dua komponen: 50 Hz dan 150 Hz. Program menggunakan FFT untuk mengeksplorasi spektrum frekuensi, menampilkan magnitude dari setiap komponen frekuensi.



Pada variasi pertama, skala magnitude diubah menjadi logaritmik (desibel), yang memungkinkan visualisasi yang lebih baik dari komponen frekuensi dengan rentang dinamis berbeda. Variasi kedua menambahkan frekuensi ketiga (300 Hz) dan menggunakan jendela Hamming untuk meminimalkan efek kebocoran spektral, memberikan representasi spektrum yang lebih halus.

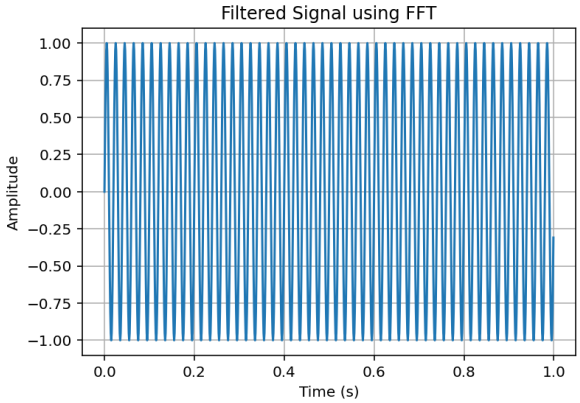


Var. 1

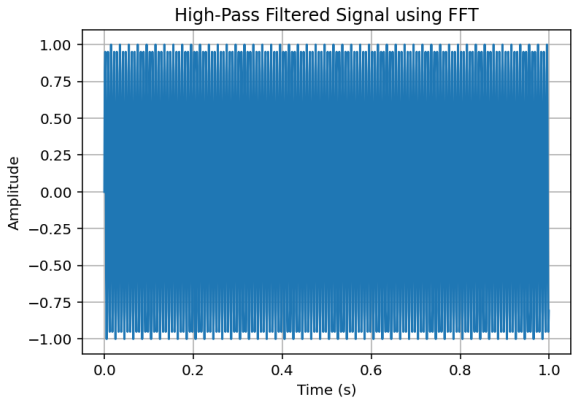


Var. 2

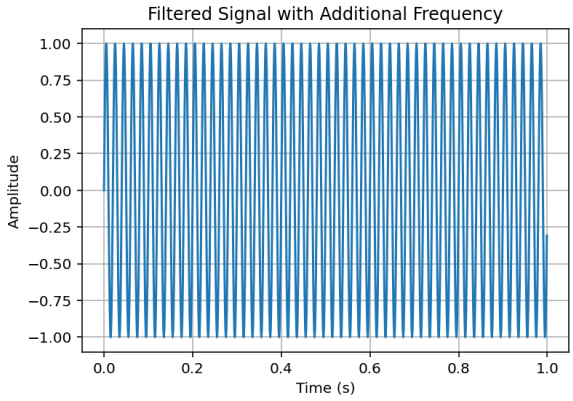
Percobaan selanjutnya membahas filtering, dengan cara mendemonstrasikan kemampuan DFT untuk memanipulasi sinyal dalam domain frekuensi. Program awal menggunakan sinyal dengan dua frekuensi (50 Hz dan 150 Hz), menerapkan low-pass filter dengan frekuensi potong 100 Hz.



Pada variasi pertama, filter diubah menjadi high-pass, menghilangkan frekuensi di bawah 100 Hz. Variasi kedua menambahkan frekuensi ketiga (250 Hz) dan menggunakan frekuensi potong 120 Hz, menunjukkan fleksibilitas filtering dalam domain frekuensi.

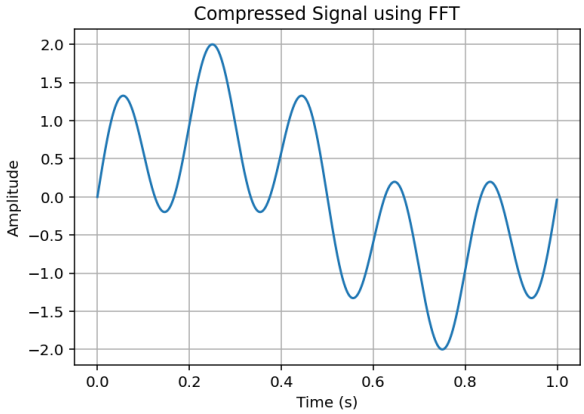


Var. 1

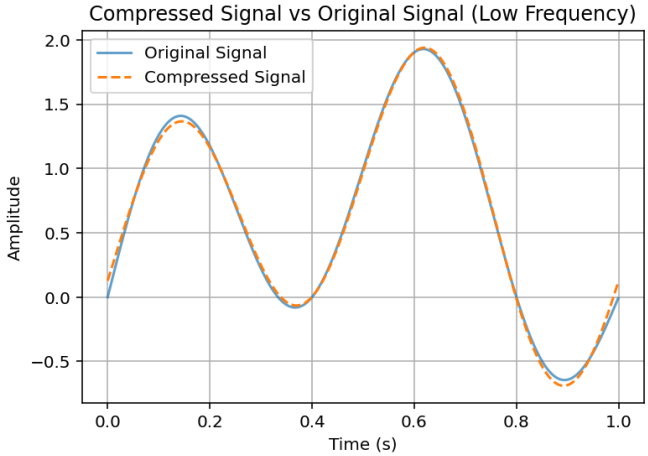


Var. 2

Pada bagian kompresi sinyal, praktikum menggunakan teknik pengurangan koefisien frekuensi. Program awal menggunakan sinyal dengan dua frekuensi rendah (1 Hz dan 5 Hz), menerapkan kompresi dengan menghapus koefisien di bawah ambang tertentu.



Pada variasi pertama, frekuensi diubah menjadi 0,5 Hz dan 2 Hz, dengan plot yang membandingkan sinyal asli dan terkompresi. Hal ini menunjukkan bagaimana kompresi dapat mempertahankan karakteristik utama sinyal sambil mengurangi kompleksitas data.



4. KESIMPULAN

Penerapan DFT dalam praktikum ini memperlihatkan kemampuan transformasi sinyal diskrit dari domain waktu ke domain frekuensi dengan menggunakan berbagai metode komputasi. Hasil percobaan mengonfirmasi karakteristik teoritis DFT, meliputi representasi frekuensi, kompleksitas komputasional, dan fleksibilitas dalam analisis sinyal. Melalui implementasi numerik, praktikum berhasil menunjukkan bagaimana algoritma FFT dapat secara efisien mentransformasikan sinyal, memvalidasi keunggulan metode ini dalam menganalisis dan memanipulasi data digital dengan kompleksitas komputasi yang rendah. Kesimpulannya, perubahan parameter sinyal dalam proses transformasi memiliki dampak signifikan terhadap representasi spektral dan efisiensi komputasi